



Nuovo Mondo



Limonta, Lecchi & Pagnoncelli

</ Indice

{01}

Testo del problema

{02}

Overview della soluzione

{03}

Struttura del codice

{04}

Costanti e variabili di stato

{05}

Funzioni per modulo

{06}

Game loop e ciclo di viaggio

{07}

Test eseguiti

{08}

Conclusioni

</ Qual è il problema?

Obiettivo: Riadattare in Python il gioco di simulazione Nuovo Mondo pubblicato da DeAgostini su Il Mio Computer (1984/1985).

Requisiti della riadattazione:

- Fase di allestimento: arruolamento equipaggio (5 ruoli), acquisto scorte (4 categorie) e merci di partenza (6 tipi)
- Viaggio simulato a settimane con 11+ eventi casuali ramificati
- Sistema di morale individuale per ogni membro dell'equipaggio (0–100)
- Baratto con gli indigeni al Nuovo Mondo per ottenere le merci rare (perle, manufatti, spezie, diamanti)
- 4 finali possibili in base al saldo finale rispetto al capitale di 2000 monete
- Salvataggio persistente delle statistiche su file JSON
- Possibilità di ricominciare con una nuova spedizione

python>



</ Overview della soluzione

} /> [

- Linguaggio: Python 3.12
- Librerie: termcolor (colori), random (casualità), json (salvataggi), os, sys, time
- Strumenti: VS Code, Canva
- Fonti: documentazione Python ufficiale, appunti di classe
- Nessuna classe usata: tutto basato su dizionari, funzioni, while loop, if/elif/else, try/except

</ Struttura del codice (moduli)

Modulo	Responsabile	Contenuto
ciclo_gioco.py	Limonta	Entry point, menu principale, fasi di gioco, stato iniziale
nuovo_mondo.py	Limonta/Pagnoncelli	I/O, salvataggi JSON, stampa, game over base, ciclo viaggio
Stati_Ingaggio.py	Pagnoncelli	Costanti, equipaggio, scorte, merci, fasi allestimento
Morale_Eventi.py	Limonta	Morale, ammutinamento, 15 eventi casuali, gestione evento casuale
epilogo.py	Lecchi	Riepilogo settimanale, barre progresso, ciclo viaggio dettagliato
riepilogo_gg.py	Lecchi	Profitti, resoconto, asta nave, finali
game_over.py	Lecchi	Game over specifici (epidemia, pirati, indigeni, tradimento)
tradimento_event.py	Lecchi	Evento tradimento ramificato
Indigeni_baratto.py	Lecchi	Primo contatto, esplorazione, cerimonia, baratto

ciclo_gioco → nuovo_mondo ↔ Morale_Eventi ↔ Stati_Ingaggio → Indigeni_baratto, epilogo, riepilogo_gg, game_over, tradimento_event

01101110 01110101 01101111 01110110 01101111 00100000 01101101 01101111 01101110 01100100 01101111

</ Costanti e variabili di stato

Costanti (da Stati_Ingaggio.py):

- COSTI_RUOLO — costo/settimana per ruolo (cuoco 15, marinaio 10, meccanico 15, medico 25, navigatore 20)
- NOMI_RUOLO — dizionario codice → nome visualizzato
- COSTI_SCORTE — verdura 0.5, frutta 1.0, carne 2.0, acqua 0.5 (per kg/brl)
- CONSUMI_SETTIMANALI_PER_MEMBRO — consumo base a settimana per categoria
- COSTI_MERCI — medicinali 30, armi 50, sale 20, stoffa 25, coltelli 15, diamanti 200

Variabili di stato (dizionario stato creato da crea_stato_iniziale() in ciclo_gioco.py):

- stato["budget"] = 2000 monete iniziali · stato["spese_iniziali"]
- stato["equipaggio"] — numero membri per ruolo · stato["morale_individuale"] — morale 0–100 per nome
- stato["scorte"] — verdura, frutta, carne, acqua · stato["razioni_moltiplicatore"]
- stato["merci"] — 6 tipi · stato["risorse_baratto"] — perle, manufatti, spezie
- stato["punti_ammutinamento"] · stato["integrita"]
- stato["settimane_percorse"] · stato["settimane_extra"] · stato["settimane_risparmiate"]
- stato["albatro_ucciso"] · stato["avvistamenti_albatro"]
- stato["fase"] — traccia il punto di avanzamento (inizio / arruolamento / provviste / merci / andata / nuovo_mondo / ritorno)
- stato["esito"] — "In corso" durante la partita; aggiornato al termine



</ Funzioni: nuovo_mondo.py

(Limonta/Pagnoncelli)

Funzione	Descrizione
<code>pulisci_schermo()</code>	<code>cls</code> su Windows, <code>clear</code> su Linux/Mac
<code>carica_dati()</code>	Legge <code>galeone_save.json</code> ; gestisce file mancante o corrotto con <code>try/except</code>
<code>salva_dati(dati)</code>	Scrive su <code>galeone_save.json</code> con <code>json.dump</code> , encoding UTF-8
<code>archivia_partita(capitano, stato, esito)</code>	Chiede un nome, salva la partita nel JSON con l'esito
<code>mostra_statistiche_globali(dati)</code>	Stampa morti, vittorie epiche, vittorie di Pirro, rovine
<code>leggi_input(prompt_testo)</code>	Input carattere per carattere con supporto ESC (→ <code>InterruptedError</code>) e backspace
<code>chiedi_opzione(prompt_testo)</code>	Chiede S/N, ripete finché non valido
<code>chiedi_scelta(prompt_testo, opzioni_valide)</code>	Ripete la domanda finché la scelta è valida
<code>stampa_lenta(testo, colore, attrs, ritardo)</code>	Effetto typewriter; Invio salta l'animazione
<code>game_over(messaggio, stato, capitano)</code>	+1 morti nelle stats, schermata rossa, archivia
<code>variazione_stat(messaggio, colore)</code>	Stampa riga feedback indentata e colorata
<code>_ciclo_viaggio(stato, capitano, fase_nome, settimane_base)</code>	Loop settimanale: morale basso → +settimane; evento ogni 2 sett.; consuma scorte; controlla ammutinamento; verifica game over

</ Funzioni: ciclo_gioco.py

(Limonta)

Funzione	Descrizione
introduzione()	Schermata iniziale, narrazione ambientata a Siviglia 1519, chiede il nome del capitano; restituisce (capitano, 2000)
viaggio_andata(stato, capitano)	Controlla se manca il cuoco (+30 PA); chiama _ciclo_viaggio(..., "ANDATA", 8)
viaggio_ritorno(stato, capitano)	Rifornisce le scorte per 3 settimane (dono del capo tribù); -1 settimana se c'è un navigatore; +1 settimana se albatro ucciso; chiama _ciclo_viaggio(..., "RITORNO", n)
crea_stato_iniziale()	Restituisce il dizionario base dello stato con budget=2000 e tutti i campi inizializzati a zero
normalizza_stato(stato)	Aggiunge i campi mancanti per compatibilità con salvataggi precedenti
esegui_partita(nuova, dati_salvati)	Coordina tutte le fasi in sequenza tramite stato["fase"]; gestisce InterruptedException (ESC) → salvataggio rapido
menu_principale()	Loop: 1 Nuova partita / 2 Continua (filtra solo esito "In corso") / 3 Statistiche / 4 Esplora archivi / 0 Esci

01101110 01110101 01101111 01110110 01101111 00100000 01101101 01101111 01101110 01100100 01101111

</ Funzioni: Stati_Ingaggio.py

Utility equipaggio e scorte

(Pagnoncelli)

Funzione	Descrizione
<code>stampa_risorse(stato)</code>	Dashboard: equipaggio per ruolo, budget, scorte, morale medio, integrità, punti ammutinamento
<code>variazione_stat(messaggio, colore)</code>	Stampa feedback colorato indentato (usata da più moduli)
<code>conta_equipaggio(stato)</code>	Somma tutti i valori di <code>stato["equipaggio"]</code>
<code>aggiungi_membro(stato, ruolo, nome)</code>	Aggiunge membro con morale=100; nome progressivo automatico (es. Cuoco_2)
<code>rimuovi_membro(stato, ruolo)</code>	Rimuove il membro con morale più bassa del ruolo; aggiorna debito
<code>controlla_morti_morale_zero(stato)</code>	Elimina i membri con morale ≤ 0 e decrementa equipaggio e debito
<code>incrementa_settimane(stato, n=1)</code>	Incrementa <code>stato["settimane_percorse"]</code>
<code>consuma_scorte_dettagliate(stato, moltiplicatore)</code>	Consuma scorte per categoria $\times$ equipaggio $\times$ moltiplicatore $\times$ razione; applica -10 morale se esaurite; restituisce "ok" / "ammutinamento" / "affondato" / "scorte_esaurite"

01101110 01110101 01101111 01110110 01101111 00100000 01101101 01101111 01101110 01100100 01101111

</ Funzioni: Stati_Ingaggio.py

Fasi di allestimento

(Pagnoncelli)

Funzione	Descrizione
fase_arruolamento(stato, capitano)	Taverna: 5 ruoli obbligatori + marinai extra; max 16 membri; paga differita; blocca se mancano ruoli; restituisce True/False
fase_acquisto_provviste(stato, capitano)	Mercato: 4 categorie con quantità suggerita per 16 settimane; gestione razioni per categoria (×0.5 o ×2); restituisce True/False
fase_merci_arsenale(stato, capitano)	Arsenale: 6 merci; traccia spesa sessione; avviso se nessuna arma; restituisce True/False
normalizza_stato_ingaggio(stato)	Aggiunge campi mancanti per compatibilità con salvataggi precedenti (versione locale, separata da ciclo_gioco.normalizza_stato)

01101110 01110101 01101111 01110110 01101111 00100000 01101101 01101111 01101110 01100100 01101111

</ Funzioni: Morale_Eventi.py

Morale e ammutinamento

(Limonta)

Funzione	Descrizione
varia_morale_tutti(stato, delta, motivo)	Modifica morale di tutti i membri (clamp 0–100); stampa feedback; chiama <code>controlla_morti_morale_zero</code>
controlla_morti_morale_zero(stato)	Rimuove i membri con morale ≤ 0 ; aggiorna equipaggio
equipaggio_basso_morale(stato, soglia=30)	True se $>50\%$ dell'equipaggio ha morale $\leq$ soglia
calcola_ammutinamento(stato)	6 criteri cumulativi; restituisce (punti, lista_cause); game over se ≥ 100
aggiungi_punti_ammutinamento(stato, punti, motivo)	Accumula punti ammutinamento nello stato con log colorato
perdita_frazione(valore)	Calcola perdita casuale: 50%, 33%, 25% o 20% del valore dato
step2_controllo_scorte(stato, settimane_rimanenti)	Verifica scorte sufficienti per le settimane restanti; stampa avvisi per categoria
controlla_morale_basso(stato)	Verifica e stampa avviso se più della metà dell'equipaggio è sotto soglia 30
gestisci_evento_casuale(stato)	Pesca a caso tra i 15 eventi disponibili e lo esegue; evita ripetizioni con <code>eventi_accaduti</code>

</ Funzioni: Morale_Eventi.py 18 eventi casuali (Limonta)

Funzione	Effetto	Funzione	Effetto
evento_uomo_in_mare	Membro casuale muore; −15 morale tutti; +10 PA	evento_ondata	Perdita frazione armi
evento_verdura_in_mare	Perdita frazione verdura; −10 morale	evento_infestazione_ratti	Perdita frazione stoffa
evento_frutta_in_mare	Perdita frazione frutta; −10 morale	evento_albatro	Senza armi: solo avvistamento; con armi: tiri 50/50; se abbattuto: +carne +10 morale, ma albatro_ucciso=True → +30 PA (maledizione)
evento_carne_in_mare	Perdita frazione carne; −10 morale	evento_scialuppa	Naufraghi avvistati: se si recuperano, +1–3 membri con morale casuale
evento_acqua_in_mare	Perdita frazione acqua; −10 morale	evento_epidemia	Senza medicinali: morti; con medicinali: si consuma 1 bottiglia, −morale; può chiamare game_over_epidemia
evento_pesca_miracolosa	+8–20 kg carne; +5 morale tutti	evento_pirati	Senza armi: sconfitta certa; con armi: combattimento a turni con dadi; perdita armi e possibili morti; può chiamare game_over_pirati
evento_tempesta_miracolosa	+10–25 brl acqua; +15 morale tutti	evento_timone	Danno al timone: −integrità; meccanico presente riduce il danno
evento_venti_favorevoli	−1 settimana viaggio; +5–15 morale casuale	evento_vento	Navigatore presente: −1 settimana; senza: +1 settimana
evento_cattivo_tempo	Perdita frazione medicinali	evento_isola	50%: approdo sicuro (+scorte); 50%: indigeni ostili (possibile game over o perdita merci)

</ Funzioni: epilogo.py

(Lecchi)

Funzione	Descrizione
<code>_barra_progresso(valore, massimo, lunghezza=20)</code>	Genera stringa visiva colorata in verde/giallo/rosso per visualizzare le percentuali
<code>_colore_soglia(valore, soglia_ok, soglia_warn)</code>	Restituisce "green" / "yellow" / "red" in base alle soglie
<code>riepilogo_fine_settimana(stato, settimana, settimane_totali, fase_nome)</code>	Dashboard completa a fine settimana: avanzamento viaggio, integrità nave, punti ammutinamento, scorte per categoria con barre e settimane coperte, morale individuale di ogni membro, stiva merci, finanze (budget + debito stimato), avvisi speciali (albatro, nessun cuoco/medico/navigatore, integrità critica)
<code>_ciclo_viaggio(stato, capitano, fase_nome, settimane_base)</code>	Versione estesa del ciclo: morale basso → +settimane; evento ogni 2 settimane; chiama <code>riepilogo_fine_settimana</code> ; consuma scorte; controlla ammutinamento; verifica tutti i game over

</ Funzioni: riepilogo_gg.py

(Lecchi)

Funzione	Descrizione
<code>calcola_profitto_finale(stato)</code>	$\text{perle} \times 180 + \text{manufatti} \times 250 + \text{spezie} \times 140 + \text{diamanti} \times 500$
<code>mostra_resoconto(capitano, stato, profitto)</code>	Stampa riepilogo: equipaggio sopravvissuto, settimane, integrità, ammutinamento, risorse ottenute, profitto totale
<code>asta_nave(debito)</code>	Loop offerte casuali 50–1200 monete; il capitano accetta/rifiuta; offerte non ripetibili; True se il debito viene saldato
<code>conclusione(capitano, stato)</code>	Calcola saldo finale (profitto + budget – debito) → assegna il finale → aggiorna stats → archivia

01101110 01110101 01101111 01110110 01101111 00100000 01101101 01101111 01101110 01100100 01101111

</ Funzioni: game_over.py

(Lecchi)

Funzione	Descrizione
game_over(messaggio, stato, capitano)	Generico: +1 morti nelle stats, schermata rossa con messaggio, archivia la partita
controlla equipaggio_vivo(stato, capitano)	Se equipaggio = 0 → game over "tutti morti"
game_over_epidemia(stato, capitano)	Game over specifico: epidemia sterminante
game_over_pirati(stato, capitano)	Game over specifico: massacro pirata
aggiorna_morale_e_controlla_morti(stato, capitano)	Rimuove i morti per morale zero; se equipaggio = 0 → game over
game_over_indigeni(stato, capitano)	Game over specifico: attacco indigeni
game_over_tradimento(stato, capitano)	Game over specifico: tradimento scoperto dal capo tribù
asta_nave(debito)	Versione alternativa dell'asta (parallela a riepilogo_gg.asta_nave)
valuta_finale(capitano, stato, profitto, soldi_residui)	Calcola saldo e assegna il finale — versione alternativa usata da game_over.py

</ Funzioni: tradimento_event.py

(Lecchi)

Funzione	Descrizione
<code>_dado(facce=6)</code>	Lancia un dado a N facce; usato nel duello
<code>_pausa() / _stampa_separatore() / _stampa_intestazione_evento()</code>	Utility di presentazione dell'evento
<code>_controlla_equipaggio(stato, capitano, soglia=3)</code>	Blocca l'evento se equipaggio < soglia (troppo pochi per un tradimento)
<code>_fase_scoperta(capitano, traditore, ruolo, arma, motivazione)</code>	Narra la scoperta: chi è il traditore, che arma ha, qual è la sua motivazione
<code>_fase_scelta_iniziale()</code>	Il capitano sceglie: [A] Affronta / [S] Spia / [F] Fuggi
<code>_ramo_affronta → _duello / _perdona</code>	Affronta → duello con dadi (vittoria/ferita/morte capitano) oppure perdono con rischio recidiva
<code>_ramo_spia → _svela_piano / _leva_potere</code>	Spia → smascheramento pubblico oppure uso del traditore come agente doppio
<code>_ramo_fuggi → _contrattacco_tardivo / _abbandono_nave</code>	Fuga → contrattacco tardivo oppure abbandono nave (game over)
<code>_epilogo(stato, ..., esito_positivo)</code>	Conclusione narrativa con effetti su morale e ammutinamento
<code>evento_tradimento(stato, capitano)</code>	Entry point: sceglie il traditore casuale tra i vivi e avvia la sequenza

01101110 01110101 01101111 01110110 01101111 00100000 01101101 01101111 01101110 01100100 01101111

</ Funzioni: Indigeni_baratto.py

(Lecchi)

Funzione	Descrizione
<code>_inizializza_relazione_indigeni(stato)</code>	Crea stato["relazione_indigeni"] (0–100)
<code>_varia_relazione(stato, delta, motivo)</code>	Modifica la relazione con log colorato
<code>_stampa_stato_relazione(stato)</code>	Mostra il livello di relazione attuale
<code>fase_primo_contatto(stato, capitano)</code>	Primo incontro: 3 scelte diplomatiche che influenzano la relazione
<code>fase_doni_iniziali(stato, capitano)</code>	Offerta di merci come dono → migliora la relazione
<code>fase_esplorazione_villaggio(stato, capitano)</code>	Scelta di 4 luoghi: giardino erbe (+medicinali), sorgente (+acqua), cucine (+cibo), tempio (rischio/ricompensa alta)
<code>fase_cerimonia_tribale(stato, capitano)</code>	Cerimonia: la relazione determina gli esiti positivi o negativi
<code>fase_baratto_indigeni(stato, capitano)</code>	Baratto sale/stoffa/coltelli/diamanti contro perle/manufatti/spezie; prezzi dipendono dalla relazione
<code>evento_tradimento_indigeni(stato)</code>	Se relazione bassa: tradimento → perdita merci, morale e possibile game over
<code>arrivo_nuovo_mondo(stato, capitano)</code>	Coordina tutte le fasi in sequenza: contatto → doni → esplorazione → cerimonia → baratto

01101110 01110101 01101111 01110110 01101111 00100000 01101101 01101111 01101110 01100100 01101111

</ Game loop completo

Flusso dell'esecuzione dall'avvio alla fine:

1. menu_principale() — 1 Nuova / 2 Continua / 3 Statistiche / 4 Archivi / 0 Esci
2. esegui_partita() — controlla stato["fase"] e riprende dal punto giusto
3. introduzione() → nome capitano, budget 2000 monete
4. fase_arruolamento() → fase_acquisto_provviste() → fase_merci_arsenale()
5. viaggio_andata() → _ciclo_viaggio("ANDATA", 8): per ogni settimana → morale basso?
+sett → evento ogni 2 sett → riepilogo → consuma scorte → ammutinamento → game over?
6. arrivo_nuovo_mondo() → primo contatto → doni → esplorazione → cerimonia → baratto
7. viaggio_ritorno() → scorte regalo 3 sett → navigatore -1 sett → albatro +1 sett →
_ciclo_viaggio("RITORNO", n)
8. conclusione() → profitto → resoconto → saldo → finale → salva JSON
9. ESC in qualunque punto → InterruptedException → salvataggio rapido → ritorno al menu

</ Il file JSON

Il file contiene due sezioni principali:

- stats — statistiche globali aggiornate a ogni partita: morti · vittorie_epiche · vittorie_pirro · rovine
- salvataggi — dizionario nome → {capitano, stato}: Ogni salvataggio contiene l'intero dizionario stato con tutti i campi, incluso stato["fase"] che permette di riprendere esattamente dal punto di interruzione e stato["esito"] che vale "In corso" per le partite attive e il finale per quelle terminate.

Esempio dai test reali (da galeone_save.json caricato): partita "Sara" in fase ritorno con 35 perle, 2.5 manufatti, relazione indigeni 67, eventi accaduti: infestazione ratti, epidemia, ondata → Vittoria Epica.

</ Test eseguiti

- **Test dei 4 finali:** Vittoria Epica, di Pirro, Nave Venduta, Rovina Totale
- **Test dei 6 game over specifici:** morale zero, ammutinamento, naufragio, epidemia, pirati, indigeni ostili, tradimento letale
- **Test ESC in ogni fase:** salvataggio rapido e ripresa corretta tramite stato["fase"]
- **Test dei 15 eventi casuali,** inclusi rari: albatro con/senza uccisione, scialuppa, isola
- **Test viaggio ritorno:** bonus navigatore (−1 sett), malus albatro (+1 sett), rifornimento scorte
- **Test salvataggio/caricamento JSON e file corrotto** (gestito con try/except)
- Test con utenti reali: verificato su partite archiviate nel JSON (Das, Sara, test1, test2)
- Nessun bug bloccante riscontrato a fine test end-to-end

</ Conclusioni

Risultato: Gioco funzionante con 9 moduli, 80+ funzioni, 15 eventi casuali, viaggio di andata e ritorno, sistema diplomatico con gli indigeni, evento tradimento ramificato a 3 rami, 4 finali distinti, salvataggio persistente con ripresa dalla fase esatta.

Miglioramenti possibili:

- **Classi** (Capitano, Membro, Nave) per ridurre la dipendenza dal dizionario stato
- **Mappa testuale del viaggio** con avanzamento visivo settimana per settimana
- **Prezzi di baratto** variabili in base a domanda/offerta degli indigeni
- **Gestione multiplatforma** completa anche su Linux/Mac per leggi_input

</ Grazie!

} /> [

/>